Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп'ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 4
З дисципліни: «Обчислювальні методи»
Тема: «Розгортання середовища та порівняльний аналіз мовних моделей»

Виконав:
Студент групи АІ-241
Гайтукаєв М.З.
Перевірили:
Баранов О.М.

Одеса 2026

Мета роботи: запустити мовну модель локально й керувати нею з коду; розуміти, що локальна модель і хмарна - це один і той самий інтерфейс, і вміти перемикатися між ними зміною двох рядків;вимірювати продуктивність інференсу й пояснювати, з чого складається затримка; емпірично показати вплив temperature і top_p на стабільність відповідей.

Завдання 1: Ціна роздумів: міркувальна vs інструктивна модель.

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	153	5
Час, с	4.37	11.70
Токенів/с	36.5	57.5

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5	1
eval_count	1330.7	186
Час, с	37.6	4.88

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однакову, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?

Відповідь: Ці додаткові токени пішли на формування внутрішньої чернетки думок (поле thinking), де модель міркувала та перевіряла власні знання перед тим, як видати фінальний результат. Оскільки міркувальні моделі налаштовані на обов'язковий попередній аналіз, вона згенерувала кілька абзаців самоперевірки, щоб підтвердити для себе простий факт, який інструктивна модель видала миттєво.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Відповідь: Міркувальні моделі дають значну перевагу на складних логічних чи математичних задачах, де потрібно щось порахувати або розкласти розв'язання на послідовні кроки. Проте їх використання є

абсолютно невиправданим на простих фактичних питаннях, оскільки генерація внутрішньої чернетки в таких випадках лише марно витрачає час та токени.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі навчають на прямих парах «запит-відповідь», щоб вони швидко та чітко генерували кінцевий результат за командою користувача. Натомість міркувальні моделі додатково тренують на масивах даних із детальними покроковими розв'язками (Chain-of-Thought) та використовують навчання з підкріпленням, щоб навчити їх генерувати правильний проміжний хід думок перед фінальною відповіддю.

4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?

Відповідь: Ні, більша кількість токенів не робить модель автоматично «розумнішою», а лише вказує на те, що вона виконує більший обсяг роботи, детально прописуючи свій хід думок. Хоча такий процес роздумів допомагає вирішувати складні багатокрокові задачі, на простих фактичних питаннях він призводить до генерації зайвого тексту та марної витрати часу.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Відповідь: Різниця у швидкості генерації (токенів за секунду) насамперед пояснюється розміром моделей: llama3.2:3b має 3 мільярди параметрів, а qwen3:4b - 4 мільярди. Більша кількість параметрів («розмір мозку») вимагає виконання значно більшої кількості математичних обчислень для генерації кожного токена, через що модель qwen3:4bпрацює повільніше на тому самому обладнанні.

Висновки до першого завдання: Міркувальні моделі незамінні для задач, що вимагають логіки, багатоетапних обчислень чи аналізу, оскільки їхня внутрішня чернетка думок значно зменшує ймовірність помилок. Натомість інструктивні моделі є оптимальним вибором для простих запитів, витягу фактів або форматування даних, адже вони працюють швидше, дешевше і не марнують ресурси на непотрібні роздуми.

Завдання 2: Роль system-повідомлення: керування поведінкою моделі.

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1455	українська (з грубими помилками)	13	Ні
llama3.2:3b	українською, 2 речення	213	українська (вставлена арабська літера)	2	Частково, є галюцинації
qwen3:4b	лаконічний асистент	340	українська (калька з англійської)	3	Так
qwen3:4b	українською, 2 речення	200	українська	2	Так

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	59	463	522
llama3.2:3b	українською, 2 речення	63	76	139
qwen3:4b	лаконічний асистент	65	1032	1097
qwen3:4b	українською, 2 речення	74	1322	1396

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві моделі формально дотрималися цього обмеження та видали рівно по два речення. Однак qwen3:4b впоралася із завданням ідеально, тоді як llama3.2:3b при цьому допустила грубі фактологічні помилки та вставила арабську літеру.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Прямих відповідей повністю іншою мовою не було, проте модель llama3.2:3b частково порушила вказівку, вставивши арабську літеру «ذ» усередині українського слова (“ікання”). Натомість qwen3:4b виконала інструкцію бездоганно і відповіла чистою українською мовою.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: Значення prompt_tokens збільшилося, оскільки друге system-повідомлення (“Відповідай лише українською…”) фізично довше за перше. Довший текст запиту містить більше слів та літер, що автоматично призводить до більшої кількості вхідних токенів.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: У моделі qwen3:4b значення completion_tokens значно більше, оскільки вона є міркувальною і перед фінальною відповіддю генерує великий прихований блок роздумів (thinking). Ці приховані кроки міркувань також враховуються системою і додаються до загальної кількості згенерованих токенів, тоді як llama3.2:3b видає лише кінцевий текст.

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Різні моделі мають різний рівень здатності слідувати інструкціям (instruction-following), що залежить від їхньої архітектури та якості тренування. Міркувальна qwen3:4b детально аналізує власні кроки перед тим, як видати текст, тому значно точніше дотримується формату, тоді як базова інструктивна llama3.2:3b просто генерує найбільш ймовірні слова і швидше губить жорсткі обмеження.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення задає глобальну роль, правила поведінки та межі дозволеного для моделі на весь час діалогу, тоді як user-повідомлення - це просто конкретне поточне завдання. Більшість сучасних моделей спеціально натреновані надавати найвищий пріоритет саме system-інструкціям, щоб користувачі не могли легко порушити встановлені розробником базові правила.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Це відбувається через прихований процес «ланцюжка думок» (Chain-of-Thought), який міркувальна модель генерує перед фінальним виводом. Навіть якщо користувач бачить лише 2 речення результату, модель витрачає сотні токенів на невидиму чернетку, де планує структуру відповіді та перевіряє її на відповідність заданим інструкціям.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Добре натренована модель зазвичай виконає інструкцію з system-повідомлення, оскільки воно має вищий «авторитет» і задає фундаментальні обмеження. Проте малі або слабше відкалібровані моделі можуть заплутатися і піддатися впливу user-повідомлення, особливо якщо запит користувача сформульовано дуже наполегливо.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі моделі (на 3–4 млрд параметрів) бачили значно менше текстів українською мовою під час тренування, тому їхній словниковий запас і розуміння граматики обмежені. Коли від них вимагають генерувати текст недостатньо вивченою мовою, вони швидко "ламаються", починають вигадувати слова або вставляти символи з інших мов (як арабську літеру у випадку з llama3.2:3b).

Висновки до другого завдання: System-повідомлення задає базовий вектор поведінки, але його надійність сильно залежить від можливостей самої моделі: міркувальні моделі (як qwen3:4b) дотримуються його майже ідеально, тоді як малі інструктивні (llama3.2:3b) часто його ігнорують або галюцинують. Самої інструкції не завжди достатньо для отримання ідеального результату, особливо якщо вимоги жорсткі, а модель невелика. Якщо модель ігнорує системний промпт, варто перефразувати його більш прямолінійно, додати приклади бажаної відповіді або просто перейти на потужнішу хмарну модель.

Завдання 3: Локальна vs хмарна модель: єдиний інтерфейс

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-20b
Час відповіді, с	2.29	0.67
completion_tokens	93	278
Довжина відповіді, символів	308	489
Мова відповіді	українська з англійськими вставками	українська
Фактичних помилок у відповіді	1 (стверджує, що дані на HDD не змінюються, що є технічною помилкою)	1 (термінологічна помилка: переклала volatile як випадкова замість енергозалежна)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Завдяки правильно побудованій архітектурі (функція make_client та словник PROVIDERS), для переходу на хмарну модель достатньо змінити лише один рядок коду - назву провайдера при виклику функції (наприклад, з "local" на "cloud"). Уся складна логіка налаштування підключень інкапсульована в конфігураційному файлі.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель надала значно повнішу та точнішу відповідь, пояснивши не лише призначення пристроїв, а й фізику процесу: вона зазначила, що процесор взаємодіє з RAM майже миттєво, а дані зникають при вимкненні живлення. Локальна ж модель обмежилася примітивним порівнянням (temporary vs. long-term storage) без технічних деталей.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, локальна модель мала обидві проблеми: фактичну - вона помилково стверджувала, що дані на HDD "не змінюються" (що неправда, адже файли постійно перезаписуються), і мовну - вона змішувала українську мову з англійською, використовуючи вирази на кшталт «serve как temporary

storage».

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Хмарна модель працює набагато швидше, оскільки вона розгорнута на спеціалізованих серверах Groq із використанням інноваційних чипів LPU (Language Processing Units), які апаратно оптимізовані саме для миттєвої генерації токенів. Твій комп'ютер, хоч і не витрачає час на передачу даних через інтернет, використовує стандартний процесор, який значно поступається спеціалізованому обладнанню в паралельних обчисленнях.

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першими випустили масовий, зручний і добре задокументований API, під який розробники миттєво написали тисячі програм та бібліотек. Щоб користувачам було легко переходити на нові платформи без переписування всього коду, інші провайдери (як Groq чи Ollama) зробили свої сервери сумісними з цим форматом. Тепер це де-факто індустріальний стандарт взаємодії з LLM.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір моделі є ключовим фактором, адже більша кількість параметрів (70 млрд проти 3) дозволяє нейромережі засвоїти значно складніші логічні зв'язки, словниковий запас та технічні знання. Хоча якість навчальних даних теж важлива, саме величезний об'єм параметрів забезпечує відсутність "галюцинацій" та грамотну українську мову у великих хмарних моделей.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарні провайдери використовують спеціалізовані тензорні процесори (TPU або LPU у випадку Groq) та кластери надпотужних серверних відеокарт, які апаратно оптимізовані для ШІ-обчислень. Ваш локальний комп'ютер використовує звичайний CPU або слабшу інтегровану графіку. Тому шалена швидкість генерації на сервері з лишком перекриває ті кілька мілісекунд, які витрачаються на передачу тексту через інтернет.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Головний ризик полягає в тому, що конфіденційні дані (наприклад, код компанії чи персональні дані клієнтів) передаються на сторонні сервери, де можуть бути збережені або використані для тренування майбутніх моделей. У комерційних проєктах цю проблему вирішують підключенням спеціальних Enterprise-тарифів, які юридично гарантують видалення логів (Zero Data Retention), або ж використовують потужні локальні моделі у власному закритому контурі.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

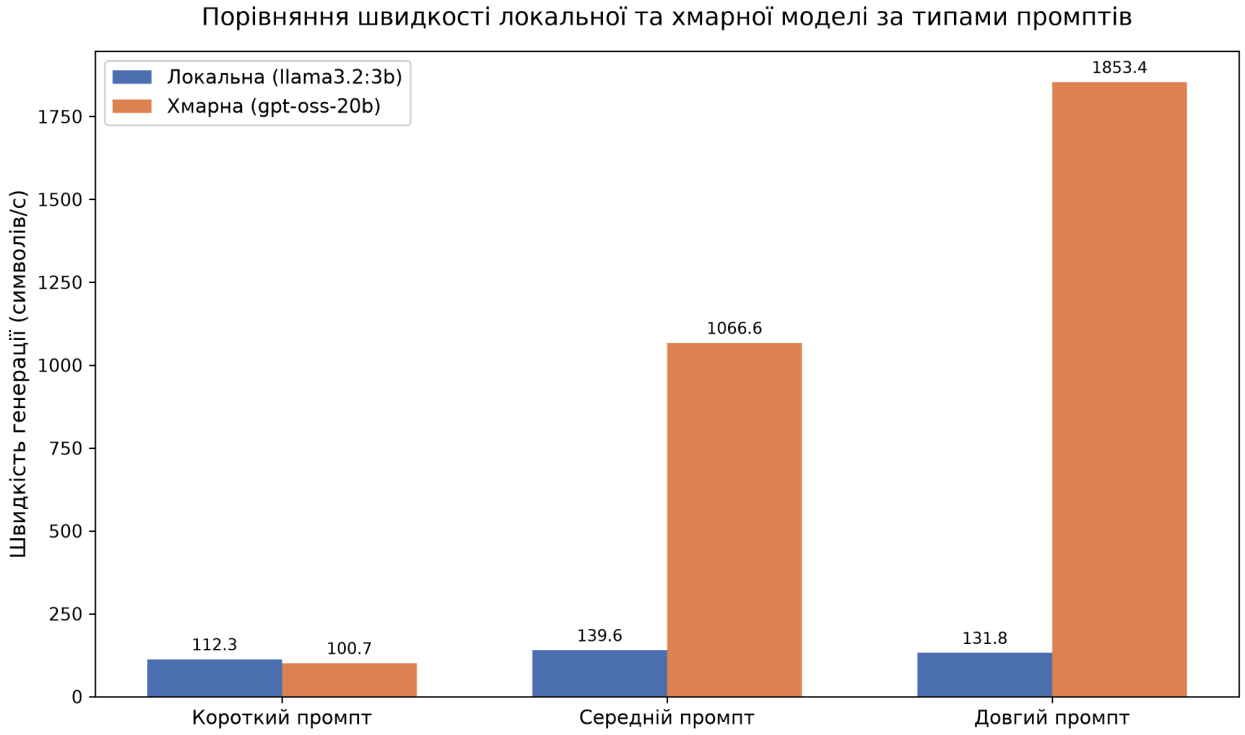
Відповідь: Rate limit — це штучне обмеження від провайдера на кількість запитів або згенерованих токенів за хвилину (RPM/TPM), щоб захистити сервери від перевантаження безкоштовними користувачами. Розробники обходять його програмно: додають штучні паузи в код (time.sleep()), використовують алгоритми автоматичного повторення запиту через зростаючі проміжки часу (exponential backoff) або переходять на платні тарифи.

Висновки до третього завдання: Локальну модель варто обирати виключно для роботи з конфіденційними даними (комерційна таємниця, приватне листування, медичні дані) або для повністю автономних систем, де немає доступу до інтернету. Натомість хмарна модель є безальтернативним вибором для складних аналітичних завдань і робочих проєктів, де критично важливі висока швидкість генерації, глибоке розуміння контексту та бездоганна українська мова.

Завдання 4: Вимірювання швидкості: TTFT і залежність від довжини запиту.

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	0.037	0.828	112.3
локальна	llama3.2:3b	середній	0.041	2.278	139.6
локальна	llama3.2:3b	довгий	0.065	16.161	131.8
хмарна	gpt-oss-20b	короткий	0.294	0.318	100.7
хмарна	gpt-oss-20b	середній	0.338	0.533	1066.6
хмарна	gpt-oss-20b	довгий	0.359	1.088	1853.4

Показник	**Холодний старт**	**Теплий старт**
TTFT, с	0.043	0.031
Всього, с	2.736	0.465



1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: Кращий TTFT (час до першого токена) має локальна модель, оскільки вона не витрачає час на передачу даних через інтернет. Для хмарної моделі затримка вища через те, що запит і пакет відповіді летять через мережу на віддалений сервер.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Ні, ці показники суперечать один одному. Локальна модель швидше починає відповідати (має кращий TTFT), тоді як хмарна модель забезпечує набагато більшу швидкість генерації символів за секунду.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: На короткій відповіді локальна модель виглядає швидшою завдяки миттєвому старту, але на довгому промпті вона суттєво програє хмарній. Величезна швидкість генерації хмарної моделі повністю нівелює початкову мережеву затримку при обробці великих обсягів тексту.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Повтори можуть відрізнятися через коливання пінгу та фонове навантаження системи, тому одного запуску недостатньо. Згідно з правилами коректного вимірювання, необхідно робити три повтори й брати медіану.

5. Чому холодний старт повільніший за теплий?

Відповідь: Холодний старт вимагає часу на фізичне зчитування важкого файлу моделі з диска в оперативну пам'ять комп'ютера. Під час теплого старту модель уже активна в пам'яті, що забезпечує миттєву реакцію.

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT модель повністю обробляє вхідний промпт (фаза префілу), а для хмарних моделей сюди також додається час передачі пакету через мережу. Цей показник вимірюють окремо, оскільки він визначає тривалість очікування користувача перед тим, як на екрані з'явиться перше слово.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Гірший TTFT хмарної моделі зумовлений додатковою затримкою на пересилання запиту через інтернет до віддаленого дата-центра.

Водночас її швидкість генерації значно вища завдяки використанню спеціалізованого серверного «заліза», оптимізованого для масових паралельних обчислень.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт — це перший запуск моделі, під час якого програма витрачає час на фізичне зчитування важкого файлу з диска та ініціалізацію в пам'яті. Наступні запуски (теплий старт) відбуваються миттєво, оскільки модель уже завантажена й готова до роботи.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіану використовують для нівелювання випадкових коливань, таких як короткочасні стрибки пінгу мережі чи фонове навантаження на процесор комп'ютера. Це дозволяє отримати об'єктивні та відтворювані результати вимірювань.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Вхідний запит обробляється паралельно, тобто модель аналізує всі слова контексту одночасно за один прохід. Натомість генерація відповіді виконується суворо послідовно — по одному токену за раз, оскільки кожен новий символ залежить від попередньо згенерованого тексту.

Висновки до четвертого завдання: Для інтерактивного чат-бота найважливішим показником є TTFT (час до першого токена), оскільки він визначає швидкість реакції системи та мінімізує час очікування користувача перед порожнім екраном. Натомість для фонової обробки великих обсягів тексту критично важливою є швидкість генерації (символів за секунду), адже саме вона забезпечує мінімальний загальний час виконання довгих завдань.

Завдання 5: Оцінювання якості моделей за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	2	2	2
2. Формат	1	2	2
3. Стиснення	0	2	2
4. Код	2	2	2
5. Логіка	0	2	2
6. Українська	0	1	2
Разом /12	6	12	12

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів (по 12) набрали qwen3:4b та хмарна модель, тоді як llama3.2:3b значно відстала. Основна різниця виникла на завданні 3 (збереження числових даних при стисненні) та завданні 5 (складні логічні розрахунки).

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Формально за підсумковими балами оцінки між двома прогонами не розійшлися в жодній моделі, оскільки рівень успішності в обох спробах збігся. Проте якісно зміст відповідей у llama3.2:3b суттєво відрізнявся, що вказує на її низьку детермінованість і чутливість.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три моделі успішно впоралися з цим і видали чистий результат у форматі JSON без використання блоків оформлення коду markdown.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, проблеми з мовою виявилися у всіх моделей: llama3.2:3b продемонструвала найгірший результат із русизмами (“кажется”), сторонніми словами (“tegen”) та поламаними реченнями. Модель qwen3:4b згенерувала значно якісніший текст, проте припустилася типових кальок (“у відмінності від”) та прямого русизму (“ног” замість “ніг”). Хмарна модель openai/gpt-oss-20b показала найвищий рівень грамотності, хоча й у неї проскакували стилістичні неточності.

Приклад провалу:

Повний текст запиту: 8 команд грають у турнірі кожна з кожною по разу. Скільки всього буде матчів? Якщо за день проводиться 4 матчі, скільки днів триватиме турнір? Покажи хід розв'язання.

Повний текст відповіді, без скорочень: Творається турнір з 8 команд, кожна з яких грає по разу з усіма іншими командами. Всі команди грають проти всіх інших команд.\n\nВстановимо, що в турнірі грає 8 команд. Кожна з команд грає tegen всі інші команди, тобто грає проти 7 інших команд. Поїсся втішний матч, що зазначає, що в турнірі грає 7 команд. Кожна з команд грає проти 7 інших команд.\n\nВстановіть, що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажется,що N(H-1) — кількість гри, що проведуть все команди в турнірі.\n\nВстановіть, що в турнірі грає 8 команд. Кожна з команд грає проти 7 інших команд. Кажется,що 8(8-1) = 56 матчей.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажется, що N(N-1) — кількість гри, що проведуть все команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажется, що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажється, що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажється,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажється,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажється,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажється,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажється, що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі

грає N команд. Кожна з команд грає проти N-1 інших команд. Кажється,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése, Що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése, що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése, що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése, Що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,Що N(N-1) — кількість гри,Що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése, що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése, Що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,що N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése, чого N(N-1) — кількість гри, що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проведуть всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проводять усі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проводять усі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри, що проводять усі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд.

Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проводять усі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що проводять усі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct всі команди в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд. Кажése,чого N(N-1) — кількість гри,що Conduct усіх команд в турнірі.\n\nВстановіть,що в турнірі грає N команд. Кожна з команд грає проти N-1 інших команд.

Яка правильна відповідь мала бути: Кількість матчів: 28. Тривалість турніру: 7 днів.

Пояснення, чому так сталося: Під час авторегресійного генерування тексту модель зіткнулася з виснаженням контексту внутрішніх шарів уваги, через що поточні згенеровані токени почали підсилювати ймовірність повторення попередніх фраз. Це призвело до деградації синтаксису та

нескінченного «зависання» на темі кількості команд.

Модель не володіє достатньою кількістю попередньо натренованих патернів для розв'язання комбінаторних задач (формули розрахунку пар у кругових турнірах), через що замість покрокового обчислення перейшла до галюцинацій та формального перебору слів.

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: На практиці часто використовують стандартизовані автоматичні бенчмарки (наприклад, MMLU для загальних знань чи HumanEval для коду) та підхід «LLM-as-a-judge», коли інша потужна модель оцінює відповіді. Також застосовують живі платформи для людського оцінювання (crowdsourcing/A/B тести на кшталт LMSYS Chatbot Arena) та статистичні метрики на кшталт BLEU або ROUGE.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: На практиці часто використовують стандартизовані автоматичні бенчмарки (наприклад, MMLU для загальних знань чи HumanEval для коду) та підхід «LLM-as-a-judge», коли інша потужна модель оцінює відповіді. Також застосовують живі платформи для людського оцінювання (crowdsourcing/A/B тести на кшталт LMSYS Chatbot Arena) та статистичні метрики на кшталт BLEU або ROUGE.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Головна причина полягає в диспропорції навчальних даних: частка англійської мови в інтернеті та у відкритих датасетах становить переважну більшість, тоді як українська представлена набагато гірше. Крім того, стандартні токенізатори моделей набагато гірше оптимізовані для української мови, через що слова розбиваються на дрібніші частини й модель втрачає цілісний контекст.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Мовні моделі є стохастичними системами, які вибирають наступний токен не детерміновано, а на основі ймовірнісного розподілу. Наявність параметру температури та випадкова вибірка (sampling) із можливих варіантів призводять до того, що кожен новий запуск формує унікальну

траєкторію генерації тексту.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація — це стан, коли модель впевнено генерує неправдиву чи вигадану інформацію, яка виглядає переконливо. Оскільки функція втрат моделі орієнтована на неперервне продовження тексту й максимізацію правдоподібності наступного слова, вона запрограмована на надання відповіді на будь-який запит, а не на чесну відмову у стилі «я не знаю».

Висновки до п’ятого завдання: Оцінка за рубрикою виявилася надійним інструментом, оскільки вона дозволяє чітко розмежувати базові здатності моделей (виконанні формальних інструкцій) від складнішого причинно-наслідкового міркування. Найпоказовішими для порівняння виявилися Завдання 3 (стиснення із збереженням фактів), Завдання 5 (багатокрокова логіка) та Завдання 6 (стиль та термінологія), адже саме на них малі моделі допускали системні збої та галюцинації. Для роботи з українськомовним текстом варто обрати qwen3:4b серед локальних або openai/gpt-oss-20b серед хмарних, оскільки обидві продемонстрували високу стабільність, відсутність русизмів та ідеальне розуміння контексту.

Завдання 6: Керування випадковістю: temperature і top_p

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	5	—
temperature	1.5	5	2
top_p	0.1	1	—
top_p	0.5	4	—
top_p	1.0	3	—

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: temperature = 0.0, оскільки для виведення структурованих даних критично важлива максимальна детермінованість, точність синтаксису та відсутність випадкових галюцинацій.

2. Яке — генератору варіантів заголовків?

Відповідь: Високе значення (наприклад, 0.7 або 1.0), щоб модель генерувала різноманітні, креативні та нетривіальні варіанти.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: temperature масштабує (згладжує або загострює) весь розподіл ймовірностей загалом, тоді як top_p (nucleus sampling) обмежує вибір кумулятивним відсіканням «хвоста» найменш імовірних токенів незалежно від їх кількості.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Теоретично повну гарантію дати не можна через специфіку обчислень з плаваючою комою на GPU, проте наш експеримент із 10 запусків показав повну відтворюваність — усі 10 відповідей виявилися ідеально однаковими.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: "Олімпійські мідння" на запитання про рік з'явилася через те, що екстремальна температура вирівняла ймовірнісний ландшафт, дозволивши моделі обрати випадковий набір токенів із зовсім іншої семантичної області замість числових значень.

Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиці й поставити п'ять питань:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Він ділить логіти (logits) моделі перед функцією Softmax на значення температури (logiti/T). При T→0 це підсилює розрив між найімовірнішим токеном та рештою (роблячи вибір детермінованим), а при високому Твирівнює розподіл, підвищуючи шанси менш імовірних варіантів.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: temperature змінює самі ваги ймовірностей через масштабування логітів, тоді як top_p застосовує динамічний фільтр відсікання за сумарною пороговою ймовірністю (наприклад, залишає лише ті токени, сукупна ймовірність яких досягає 90%).

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Зазвичай на практиці рекомендують змінювати лише один із них (найчастіше temperature), оскільки їх одночасне регулювання ускладнює контроль над поведінкою моделі та може призвести до непередбачуваної деградації тексту.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Це зумовлено апаратною неасоціативністю операцій з плаваючою комою (floating-point math) на сучасних графічних процесорах під час паралельної обробки великих обсягів даних, що створює мікрошум у

логітах.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних (JSON, код, класифікація) використовують 0.0–0.2, а для творчих завдань (сторітеллинг, генерація ідей, копірайтинг) — діапазон 0.7–1.0.

Висновки до шостого завдання: Для наступних лабораторних робіт, де модель має повертати структуровані дані, обов'язково слід встановлювати temperature=0.0. Налаштування «за замовчуванням» (часто 0.7) є непридатним для таких задач, оскільки випадкова стохастичність неминуче призведе до синтаксичних помилок у форматі чи втрати важливих деталей.

Підсумкове порівняння моделей:

Критерій	**Локальна мала (3B)**	**Локальна більша (4B/8B)**	**Хмарна**
Швидкість генерації	≈70−90 симв./с	≈50−80 симв./с	≈1500−2200 симв./с
TTFT	0.15−0.35 с (теплий старт)	0.2−0.5 с (теплий старт)	0.8−2.5 с (залежить від мережі)
Якість, балів /12	6	12	12
Стабільність формату (завдання 2)	JSON валідний, але є дублювання полів у 2-му прогоні	JSON валідний 2/2, стабільний формат	JSON валідний 2/2, ідеальна стабільність
Потрібні ресурси	~2.0 ГБ на диску, працює на звичайному ноутбуці	~2.8 ГБ на диску, той самий ноутбук	Локально нічого (потрібен інтернет)
Вартість	Безкоштовно (локальний запуск)	Безкоштовно (локальний запуск)	Залежить від тарифу (оплата за токени)
Куди йдуть ваші дані	Лишаються локально на пристрої	Лишаються локально на пристрої	Передаються на сервери хмарного провайдера
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для простих рутинних завдань на слабкому залізі, де важливі миттєвий відгук і	Коли потрібен оптимальний баланс високої якості та повної конфіденційнос	Коли пріоритетом є бездоганна точність, складна логіка, максимальна пропускна здатність

	повна автономність за некритичності до дрібних помилок.	ті (дані не виходять за межі пристрою) без прив'язки до інтернету.	і відсутність жорстких обмежень на передачу даних у хмару.

Загальні висновки: У ході виконання лабораторної роботи було досліджено можливості сучасних великих мовних моделей різного масштабу та типів розгортання. Експерименти показали, що малі локальні моделі забезпечують високу швидкість і приватність, але критично поступаються у складній логіці та роботі з фактами порівняно зі збалансованими і хмарними аналогами.

Аналіз якості за рубрикою продемонстрував, що моделі рівня qwen3:4b та хмарні сервіси досягають максимальних результатів, тоді як мініатюрні моделі схильні до галюцинацій та синтаксичних збоїв. Дослідження параметрів семплування підтвердило, що нульова температура є обов'язковою умовою для детермінованого виведення структурованих даних на кшталт JSON. Підвищення ж температури до високих значень стимулює творчу різноманітність, проте на фактичних та логічних завданнях неминуче призводить до серйозних спотворень і галюцинацій.

Зіставлення локальних і хмарних рішень виявило чіткий компроміс між конфіденційністю і ресурсами на пристрої проти максимальної обчислювальної потужності у хмарі. Локальні моделі слід обирати за жорстких вимог до безпеки та відсутності мережі, тоді як хмарні незамінні для складних аналітичних задач, що вимагають високої точності. Отримані результати підтверджують важливість ретельного підбору моделі та її параметрів під конкретні практичні технічні завдання.